

STEP SEM-3 · Week 8 Category C Practice

Answers to Quiz (MCQ) Questions and Concept (Theory) Questions

Section 1: Quiz Questions (MCQ) - Answers

Question 1

A system processes Document objects (PDFDocument, WordDocument), both implementing render(). Repeated if-else if blocks check the document type before calling render(). Which OOP principle eliminates this conditional logic?

Correct Answer: C. Polymorphism

Why: Each document type provides its own render() implementation; calling render() on a common Document reference lets the correct version run automatically via dynamic dispatch, removing the need for type-checking conditionals.

Question 2

Which scenarios represent a genuine 'is-a' relationship, making inheritance suitable? (A) A Car is a Vehicle. (B) A Rectangle is a Shape. (C) A DatabaseConnection is a NetworkResource. (D) A HelperUtility contains a Calculator.

Correct Answer: A and B

Why: A Car is a specialised kind of Vehicle, and a Rectangle is a specialised kind of Shape - true is-a relationships. A DatabaseConnection merely uses a network resource (a 'has-a'/'uses-a' relationship, not is-a), and a HelperUtility containing a Calculator is composition, not inheritance.

Question 3

A Vehicle base class has startEngine(); Car and Motorcycle override it. Iterating a list of Vehicle objects and calling startEngine() on each - what mechanism ensures the correct implementation runs for each type?

Correct Answer: C. Runtime method dispatch

Why: The JVM resolves which overridden method to call based on the object's actual runtime type, not its declared reference type - this is dynamic (runtime) method dispatch.

Question 4

A Shape base class has calculateArea(); Circle and Rectangle each provide their own implementation. Storing various Shape objects in a list and calling calculateArea() on each demonstrates what concept?

Correct Answer: C. Inheritance-based polymorphism

Why: The shared base type (Shape) combined with each subclass overriding calculateArea() is a textbook case of inheritance-based (runtime) polymorphism.

Question 5

LibraryItem has getLoanPeriod(). Book extends this method with an extra rule for new releases; DVD sets a fixed loan period. Which statements are accurate?

Correct Answer: A, C, and D

Why: (A) True - Book builds on/extends the inherited behaviour rather than replacing it entirely. (C) True - LibraryItem defines the shared contract/behaviour for all items. (D) True - a DVD object can be referenced and

processed as a LibraryItem (polymorphic reference). (B) is false - DVD's fixed loan period is specialised/overridden behaviour, not 'shared' behaviour.

Question 6

Animal has makeSound(); Dog overrides it to produce 'Woof', Cat to produce 'Meow'. Which statements are correct?

Correct Answer: A and C

Why: (A) True - calling makeSound() on an Animal reference pointing to a Dog object executes Dog's version (dynamic dispatch). (C) True - Dog's makeSound() is overridden behaviour. (B) is false - a base class method is the default/general behaviour, not 'specialised' behaviour. (D) is false - derived classes can add entirely new methods that were never declared in the base class.

Question 7

A PaymentProcessor holds a collection of Payment references (CardPayment, BankTransferPayment). What is the primary benefit of this design when adding a new WalletPayment type?

Correct Answer: C. It allows adding WalletPayment without modifying the existing PaymentProcessor's iteration logic.

Why: Because PaymentProcessor depends only on the Payment abstraction, a brand-new subclass can be added and dropped into the same collection with zero changes to the processing/iteration code - the open/closed principle in action.

Question 8

Why is using inheritance solely for superficial code reuse between unrelated classes generally an inappropriate design choice?

Correct Answer: A. It leads to tighter coupling and incorrect 'is-a' relationships, making the design rigid.

Why: Inheriting purely to reuse methods (without a genuine is-a relationship) forces an artificial hierarchy, exposes irrelevant inherited members, tightly couples the subclass to the base class's internals, and makes the design brittle and hard to change safely.

Question 9

A Notification base class has send(); EmailNotification, SMSNotification, and PushNotification each override it. What are the advantages of this design? (Select all that apply)

Correct Answer: A and C

Why: (A) True - a generic NotificationSender can send any notification type through the common reference without knowing the concrete class. (C) True - a new channel (e.g. InAppNotification) can be added without touching existing sender logic. (B) is false - logic is distributed across subclasses, not centralised in one class. (D) is false - the whole point of overriding is that each type uses a *different* send() implementation, not the exact same one.

Question 10

What are the benefits of using polymorphic collections (a list of base-class references holding derived-class objects)? (Select all that apply)

Correct Answer: A, B, and C

Why: (A) True - iteration code is written once regardless of how many concrete types exist. (B) True - each object still executes its own specialised behaviour even though it's processed uniformly. (C) True - no explicit

casting is needed since the common base type declares the method. (D) is false - polymorphic collections make it *easier*, not harder, to add new derived types.

Section 2: Concept Questions (Theory) - Answers

Question 1

Explain how inheritance enables specialized classes to reuse existing behavior from a base class and simultaneously extend or modify that behavior. Provide a concrete business example.

Inheritance lets a derived (child) class automatically acquire the fields and methods of a base (parent) class, so common logic is written once and reused everywhere, while still allowing the derived class to add new members or override existing methods to specialise behaviour for its own needs. For example, an **Employee** base class defines shared attributes like *name* and *id* and a method *calculateSalary()* that returns a basic hourly wage. A **Manager** subclass inherits the id/name handling as-is, but overrides *calculateSalary()* to add a leadership bonus on top of the base pay, and also adds a new field *teamSize*. Manager reuses everything that is common to every employee, while customising only the piece of behaviour that genuinely differs for its role.

Question 2

Describe the 'is-a' relationship in OOP. Explain why correctly identifying a genuine 'is-a' relationship is crucial when deciding to use inheritance.

An 'is-a' relationship means that an object of the derived class can be treated, both semantically and behaviourally, as an instance of the base class - for example, 'a Car is a Vehicle'. Every operation that is valid on a Vehicle should also make sense and behave correctly for a Car (this is the Liskov Substitution Principle). Identifying a genuine is-a relationship matters because inheritance should model true type hierarchies, not just be used as a shortcut for reusing code between unrelated classes (for example, making a Stack extend ArrayList purely to reuse add/remove methods). Using inheritance for a false is-a relationship exposes inappropriate methods to clients, tightly couples unrelated classes, and can break client code when a subclass is substituted for the base type but does not actually behave like one.

Question 3

Define method overriding. Using a business scenario of different Employee types calculating salary differently, illustrate how overriding lets derived classes provide their own implementations.

Method overriding occurs when a subclass provides its own implementation of a method that is already defined in its superclass, using the exact same method signature, so that the subclass's version runs instead of the parent's whenever the method is called on a subclass instance. For example, a base class **Employee** declares *calculateSalary()*. **FullTimeEmployee** overrides it to add monthly benefits on top of the base salary; **PartTimeEmployee** overrides it to compute *hoursWorked × hourlyRate*; and **Contractor** overrides it to add invoice-based project fees. A payroll system that iterates over a list of Employee references and calls *calculateSalary()* on each one automatically triggers the correct type-specific calculation for every object, without the payroll code ever needing to know each employee's concrete subtype.

Question 4

Explain the concept of runtime polymorphism (dynamic dispatch) when working with common base-type references. How does the system determine which specific method implementation to execute?

Runtime (dynamic) polymorphism means that the specific method implementation executed is decided at run time, based on the object's actual concrete type, rather than the declared type of the reference used to call it. For example, in *Animal a = new Dog(); a.makeSound();* the compiler only checks at compile time that

Animal declares a `makeSound()` method. At run time, however, the JVM looks at the actual object referred to by 'a' (a Dog instance) and invokes Dog's overridden `makeSound()` using an internal virtual method lookup (often implemented as a virtual method table per class). This lookup happens on every call, allowing one base-type reference to trigger different, type-appropriate behaviour depending on which concrete subclass object it actually points to.

Question 5

Describe how polymorphic collections work. Provide an example of a collection holding objects of a common base type but different derived types, and explain the advantages.

A polymorphic collection is a list or array declared using a base type or interface, but populated with objects of several different derived types. For example: `List<Shape> shapes = new ArrayList<>(); shapes.add(new Circle(5)); shapes.add(new Rectangle(4, 6)); shapes.add(new Triangle(3, 4, 5));`. Iterating over this list and calling `shape.calculateArea()` on each element automatically executes the correct override for that specific shape. Advantages include: (1) one simple, uniform iteration loop works regardless of how many shape types exist; (2) new shape types can be introduced later without changing the iteration/processing code (open/closed principle); and (3) there is no need for explicit type casting or repetitive if-else/switch blocks, making the code shorter, more maintainable, and less error-prone.

Question 6

Compare solving problems using polymorphism (method overriding with a common base type) versus repeated type-based conditional logic (if-else if / switch). Why is polymorphism generally preferred?

With conditional logic, a single method inspects an object's type tag and branches into type-specific code (e.g. `if (type.equals("CARD")) ... else if (type.equals("WALLET")) ...`). Every time a new type is introduced, every such conditional block scattered across the codebase must be located and updated - a process that is repetitive and easy to get wrong, and that violates the open/closed principle. With polymorphism, each type implements its own version of the shared operation inside its own class, and client code simply calls that operation on a common base reference; dynamic dispatch resolves the correct implementation automatically. Adding a new type then only requires writing one new class - all existing call sites keep working unchanged. Polymorphism also keeps type-specific logic co-located with the data it operates on (better cohesion) and removes the risk of forgetting to update a branch somewhere, which is why it is generally preferred for managing varying behaviour among related objects.

Question 7

Explain how inheritance and polymorphism allow adding new derived types with minimal or no changes to the common processing logic. Illustrate with a new `PaymentMethod` example.

Because client code depends only on a base type or interface (for example, a **Payment** interface with a `calculateFee()` method), and dynamic dispatch resolves the correct implementation at run time, the system follows the 'open for extension, closed for modification' principle. A `PaymentProcessor` that loops over a `List<Payment>` and calls `calculateFee()` on each entry never needs to change when a new payment type is introduced. To add a new **WalletPayment** type, a developer simply creates a class `WalletPayment` implements `Payment` with its own `calculateFee()` logic, and starts adding `WalletPayment` objects into the same collection - the existing iteration and processing logic inside `PaymentProcessor` requires zero edits, because it was already written against the abstraction rather than against concrete payment classes.

Question 8

Distinguish between inherited behavior and overridden behavior. When would a derived class typically inherit a method, and when would it typically override one? Provide examples for each.

Inherited behaviour is a method (or field) that a subclass receives unchanged from its superclass and uses exactly as written, because the default implementation is already correct for that subclass. Overridden behaviour is when the subclass supplies its own implementation of a method that already exists in the superclass, because the inherited default is not appropriate for that specific subtype. For example, a base class **Employee** has a method *getEmployeeId()* that simply returns a stored ID - every subclass (Manager, Developer, Intern) typically inherits this unchanged, since ID handling does not vary by role. In contrast, *calculateBonus()* is typically overridden: Manager overrides it to compute a leadership bonus, Developer overrides it to compute a performance-based bonus, and Intern overrides it to always return a flat stipend - because each subtype genuinely needs different bonus logic.

Question 9

Explain why using inheritance solely for the sake of code reuse, when there is no genuine 'is-a' relationship, can be an inappropriate design choice. Discuss the potential negative consequences.

Using inheritance purely to borrow existing code, rather than to model a true is-a relationship, forces an artificial class hierarchy: the subclass automatically inherits every method and field of the 'parent', including ones that make no logical sense for it, which misleads anyone reading the code about the real relationship between the classes. This produces tight coupling - any change to the base class can unexpectedly break the 'fake' subclass - makes the design rigid and difficult to extend safely, and can violate the Liskov Substitution Principle, causing subtle runtime bugs whenever the subclass is substituted where the base type is expected but does not actually behave compatibly (the classic example is a Stack extending ArrayList, which then exposes insert-at-arbitrary-index methods that break proper stack semantics). Favouring composition ('has-a', reusing code through delegation) over inheritance in such cases avoids these pitfalls entirely.

Question 10

Analyze a VehicleRental system with CarRental and TruckRental to identify common behaviors shared across all rentals and specialized behaviors unique to each type. Explain how inheritance models this distinction.

Both CarRental and TruckRental share behaviour and attributes common to any rental transaction: *renterName*, *rentalStartDate*, *rentalEndDate*, a base duration-fee calculation, and shared actions like *checkOut()* and *checkIn()*. These commonalities belong in a base class, say **VehicleRental**. Each rental type also has specialised behaviour: CarRental may add a per-mile charge and require a standard driver's licence class, while TruckRental may add a cargo-weight surcharge, a commercial-licence requirement, and different insurance rates. With inheritance, VehicleRental defines the shared fields/methods and declares an overridable method such as *calculateTotalFee()*; CarRental and TruckRental extend VehicleRental, automatically inheriting the shared behaviour while overriding *calculateTotalFee()* (and adding their own extra fields) to apply their specific pricing rules. This lets a central rental system process a *List<VehicleRental>* uniformly - for example, to total up revenue - while each concrete rental type still computes its own fee correctly.